

LA CANTINA
Object Design Document
Versione 1.0



Progetto: LA CANTINA	Versione: 1.0
Documento: Object Design Document	Data: 22/12/2025

Coordinatore del progetto:

Nome	Matricola
GRAZIOSO Fabrizio	0512122950

Partecipanti:

Nome	Matricola
LIGUORI Alessandro	0512119887
CICALESE Gabriele	0512116443
GRAZIOSO Fabrizio	0512122950

Scritto da:	GRAZIOSO Fabrizio
--------------------	-------------------

Revision History

Data	Versione	Descrizione	Autore
22/12/2025	1.0	Scrittura documento	Grazioso, Cicalese, Liguori

Sommario

1.	Introduzione	4
1.1.	Object design trade-offs	4
1.2.	Interface documentation guidelines	4
1.3.	Definitions, acronyms, and abbreviations	5
1.4.	References	5
2.	Packages	5
2.1.	It.lacantina.model	5
2.2.	It.lacantina.model.dao	5
2.3.	It.lacantina.control	5
2.4.	It.lacantina.bean	5
3.	Class Interfaces	6
3.1.	Sottosistema UserManagement	6
	Class: UtenteDAO	6
	Class: AutenticazioneControl (Servlet)	6
3.2.	Sottosistema CatalogManagement	6
	Class: ProdottoDao	6
3.3.	Sottosistema Cart	6
	Class: Carrello	6
3.4.	Sottosistema OrderManagement	7
	Class: OrdineDAO	7
	Class: CheckoutControl (Servlet)	7
3.5.	Sottosistema Administration	7
	Class: AdminProdottoDAO	7
	Class: AdminOrdineDAO	7

1. Introduzione

1.1. Object design trade-offs

Durante la definizione del design degli oggetti per il sistema "La Cantina", il team di sviluppo ha dovuto affrontare diverse decisioni architetturali (trade-offs) per bilanciare le prestazioni, la consistenza dei dati e la manutenibilità, in linea con i *Design Goals* definiti nel SDD

- **Buy vs. Build:** Si è optato per un approccio **Build** (sviluppo da zero) utilizzando tecnologie standard Java EE (Servlet/JSP) invece di utilizzare piattaforme e-commerce precostituite. Questa scelta è dettata dalla necessità didattica di avere il controllo completo sul codice e di implementare logiche specifiche per la "filiera corta" e il km0.
- **Memory Space vs. Response Time (Gestione Carrello):** Per la gestione del carrello (Cart), si è scelto di mantenere lo stato interamente in memoria all'interno della HttpSession del server, senza persistenza immediata sul database
 - *Trade-off:* Questa scelta sacrifica l'occupazione di memoria RAM sul server (aumentando il carico per utente attivo) per garantire tempi di risposta (Response Time) immediati durante la navigazione e l'aggiunta di prodotti, evitando continue query di scrittura al DB prima del checkout
- **Data Access Strategy (JDBC vs ORM):** È stato scelto l'utilizzo di JDBC nativo con PreparedStatement invece di framework ORM (come Hibernate)
 - *Trade-off:* Sebbene questo comporti una maggiore verbosità nel codice dei DAO, garantisce un controllo granulare sulle query SQL, ottimizzando le performance e la sicurezza (prevenzione SQL Injection), riducendo l'overhead di astrazione.

1.2. Interface documentation guidelines

Per garantire la coerenza tra i moduli sviluppati dai diversi membri del team, sono state stabilite le seguenti convenzioni di codifica e documentazione delle interfacce:

- **Nomenclatura Classi:** Le classi devono essere nominate utilizzando sostantivi al singolare in stile PascalCase (es. Prodotto, Ordine), mantenendo la lingua italiana per coerenza con il dominio
- **Nomenclatura Metodi:** I metodi devono utilizzare verbi o frasi verbali in stile camelCase.
 - I metodi dei DAO devono seguire il prefisso standard do (es. doSave, doRetrieveByKey, doUpdate).
 - I metodi dei Controller devono riflettere l'azione (es. processCheckout, login).
- **Gestione Errori:** Lo stato di errore non deve essere restituito tramite valori interi, ma sollevando eccezioni specifiche. Le eccezioni SQL (SQLException) devono essere catturate e rilanciate come eccezioni applicative (es. RuntimeException o custom DAOException) per non esporre dettagli del database al livello superiore.
- **Collezioni:** I metodi che restituiscono collezioni (es. List<Prodotto>) non devono mai restituire null, ma una lista vuota (isEmpty()) in assenza di risultati, per evitare NullPointerException.

- **Visibilità:** Tutti gli attributi delle classi Entity devono essere private. L'accesso deve avvenire esclusivamente tramite metodi pubblici get e set.

1.3. *Definitions, acronyms, and abbreviations*

- **DAO (Data Access Object):** Pattern utilizzato per astrarre e incapsulare l'accesso ai dati persistenti.
- **MVC (Model-View-Controller):** Pattern architetturale su cui si basa l'intero sistema.
- **Entity:** Oggetto che rappresenta un dato persistente nel dominio (es. Utente, Prodotto)
- **Session Bean:** Oggetto mantenuto nella sessione utente (es. Carrello)
- **SDD:** System Design Document.
- **RAD:** Requirements Analysis Document.

1.4. *References*

- *Requirements Analysis Document (RAD) v2.1* - La Cantina.
- *System Design Document (SDD) v2.0* - La Cantina.

2. Packages

Il codice sorgente è organizzato in package Java che riflettono la decomposizione in sottosistemi definita nel SDD e l'architettura MVC.

2.1. *It.lacantina.model*

Contiene le classi **Entity** (POJO) che rappresentano i dati del dominio. Queste classi non contengono logica di business complessa ma solo attributi e metodi di accesso.

- **Classi principali:** Utente, Prodotto, Ordine, RigaOrdine, AziendaFornitrice.
- **Dipendenze:** Nessuna dipendenza verso altri package del sistema.

2.2. *It.lacantina.model.dao*

Contiene le classi che implementano il pattern **DAO**. Gestisce la comunicazione diretta con il DBMS MySQL tramite JDBC. Questo package corrisponde al livello "Persistent data management".

- **Classi principali:** UtenteDAO, ProdottoDAO, OrdineDAO.
- **Dipendenze:** it.lacantina.model (utilizza le entity per input/output), java.sql.

2.3. *It.lacantina.control*

Contiene le **Servlet** che fungono da Controller. Questo package gestisce la logica applicativa, riceve gli input dalla View (JSP), interagisce con i DAO e il Carrello, e determina la pagina di risposta.

- **Classi principali:** AutenticazioneControl, GestioneCarrelloControl, CheckoutControl, AdminProdottoControl.
- **Dipendenze:** it.lacantina.model, it.lacantina.model.dao, it.lacantina.bean.

2.4. *It.lacantina.bean*

Contiene classi di supporto non persistenti o oggetti complessi mantenuti in sessione.

- **Classi principali:** Carrello (Rappresenta il sottosistema *Cart*).
- **Dipendenze:** it.lacantina.model (contiene oggetti Prodotto).

3. Class Interfaces

Questa sezione descrive le interfacce pubbliche delle classi chiave per i sottosistemi da implementare.

3.1. Sottosistema UserManagement

Class: UtenteDAO

Responsabile delle operazioni CRUD sulla tabella Utenti.

Operations:

- public Utente doRetrieveByEmailPassword(String email, String password)
 - *Descrizione:* Verifica le credenziali (email e hash password). Restituisce l'oggetto Utente se valido, null altrimenti.
 - *Exceptions:* SQLException
- public boolean doSave(Utente utente)
 - *Descrizione:* Registra un nuovo utente. Restituisce false se l'email è già presente.
 - *Exceptions:* SQLException

Class: AutenticazioneControl (Servlet)

Operations:

- protected void doPost(HttpServletRequest request, HttpServletResponse response)
 - *Descrizione:* Gestisce il parametro action. Se login, chiama il DAO e crea la sessione. Se logout, invalida la sessione .

3.2. Sottosistema CatalogManagement

Class: ProdottoDao

Fornisce l'accesso in lettura al catalogo.

Operations:

- public List<Prodotto> doRetrieveAll()
 - *Descrizione:* Restituisce la lista completa dei prodotti disponibili.
- public Prodotto doRetrieveByKey(int id)
 - *Descrizione:* Restituisce i dettagli di un singolo prodotto (incluso stock e descrizione).ù

3.3. Sottosistema Cart

Class: Carrello

Classe non persistente (Session Bean). Mantiene lo stato degli acquisti temporanei.

- **Attributes:**
 - private HashMap<Integer, Integer> prodotti; (Mappa ID Prodotto -> Quantità)
- **Operations:**
 - public void addProduct(int idProdotto, int quantita)
 - *Descrizione:* Aggiunge un prodotto o ne incrementa la quantità. Deve verificare che la quantità totale non superi lo stock disponibile (richiede interazione con ProdottoDAO nel Controller).

- `public void removeProduct(int idProdotto)`
 - *Descrizione:* Rimuove completamente un prodotto dalla lista.
- `public void updateQuantity(int idProdotto, int nuovaQuantita)`
 - *Descrizione:* Aggiorna la quantità di un elemento esistente.
- `public void emptyCart()`
 - *Descrizione:* Svuota il carrello dopo un ordine completato.

3.4. *Sottosistema OrderManagement*

Class: OrdineDAO

Gestisce la persistenza degli ordini e delle righe d'ordine in transazione.

- **Operations:**

- `public int doSave(Ordine ordine, Carrello carrello)`
 - *Descrizione:* Metodo transazionale.
 1. Salva la testata dell'ordine.
 2. Itera sul carrello e salva le RigaOrdine.
 3. Decrementa lo stock dei prodotti coinvolti.
 - *Return:* L'ID dell'ordine generato.
 - *Exceptions:* SQLException (effettua rollback se fallisce).
- `public List<Ordine> doRetrieveByUtente(int idUtente)`
 - *Descrizione:* Restituisce lo storico ordini per un dato utente.

Class: CheckoutControl (Servlet)

Gestisce la persistenza degli ordini e delle righe d'ordine in transazione.

Operations:

- `public void processCheckout(HttpServletRequest req, HttpServletResponse res)`
 - *Descrizione:* Verifica che l'utente sia loggato, valida i dati di spedizione, chiama OrdineDAO.doSave e infine Carrello.emptyCart().

3.5. *Sottosistema Adminitration*

Class: AdminProdottoDAO

Aggiunge funzionalità di scrittura riservate all'admin.

- **Operations:**

- `public void updateStock(int idProdotto, int newQuantity)`
 - *Descrizione:* Aggiorna la giacenza di magazzino.
- `public void updatePrice(int idProdotto, double newPrice)`
- `public void doSave(Prodotto p)`
 - *Descrizione:* Inserisce un nuovo prodotto nel catalogo.
- `public void doDelete(int idProdotto)`
 - *Descrizione:* Rimuove un prodotto dal catalogo (o soft-delete).

Class: AdminOrdineDAO

Operations:

- `public void updateStato(int idOrdine, String nuovoStato)`
 - *Descrizione:* Modifica lo stato dell'ordine (es. da "In elaborazione" a "Spedito").

- `public List<Ordine> doRetrieveAll()`
 - *Descrizione:* Recupera tutti gli ordini di tutti gli utenti per la dashboard amministrativa.